

Funciones Lambda

Clase 12

Expresión lambda

Es una función corta y sin nombre. Usada mayormente como argumento de otras funciones. Poseen la siguiente estructura.

lambda parámetros: expresión

Ejemplo:

lambda x, y: x + y

lambda x, y=1: x*y

[+Información](#)

```
def suma(x, y):  
    return x+y
```

```
def multi(x, y=1):  
    return x*y
```



Expresiones lambda

```
lista_de_acciones = [lambda x: x * 2, lambda x: x * 3]
```

```
param = 4
```

```
for accion in lista_de_acciones:
```

```
    print(type(accion))
```

```
    print(accion(param))
```



sorted(iterable, key=func, reverse=False)

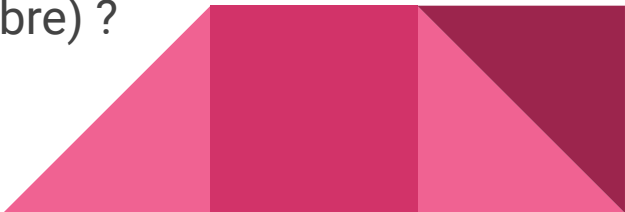
Retorna una lista ordenada del objeto iterable recibido como parámetro. Además cuenta con 2 parámetros opcionales una función (key=None) y un booleano (reverse=False).

```
lista = [(4, 1), (-6, 12), (10, 2)]
```

```
lista_ord= sorted(lista, key=lambda x: x[1])    # print(lista_ord) ?
```

```
nombres = ['Maria', 'Alberto', 'Max']
```

```
nombres.sort(key=lambda x: len(x))            # print(nombre) ?
```



map(func, iterable)

map es una función que retorna un objeto 'map' luego de aplicar una función a todos los elementos de un iterable dado.

```
lista = [(4, 1), (-6, 12), (10, 2)]
```

```
lista_suma = list(map(lambda a: a[0]+a[1], lista))
```

```
print(lista_suma)
```



map()

```
def doble(x):  
    return 2*x
```

```
lista = [1, 2, 3, 4, 5, 6, 7]
```

```
dobles1 = list(map(doble, lista))
```

```
print(dobles1)
```

```
dobles2 = list(map(lambda x: x*2, lista))
```

```
print(dobles2)
```



filter(func, iterable)

Es una función que retorna un objeto 'filter' luego de filtrar los elementos mediante una función que retorna un booleano (condición).

```
lista = [5, 6, 12, 7]
```

```
lista1 = list(filter(lambda x: x % 2 == 0, lista))
```

```
lista2 = list(filter(lambda x: x > 6, lista))
```



filter()

```
def es_par(x):  
    return x%2 == 0
```

```
lista = [1, 2, 3, 4, 5, 6, 7]  
pares1 = list(filter(es_par, lista))  
print(pares1)  
pares2 = list(filter(lambda x: x%2==0, lista))  
print(pares2)
```



Condicionales en una línea

Estructura: *valor_1* **if** *condicion* **else** *valor_2* (*if condicion else valor_3*)

Ejemplo:

```
edad = int(input('Ingrese su edad: '))
```

```
grupo_etario = "Menor" if edad < 18 else "Adulto"
```

```
print(grupo_etario)
```

```
grupo_etario = "Menor" if edad < 18 else "Adulto" if edad < 65 else "Adulto mayor"
```

```
print(grupo_etario)
```



Ejercicio 1

Utiliza una función lambda para convertir una lista de temperaturas en grados Celsius a grados Fahrenheit ($F = (C * 9/5) + 32$).

```
celsius_temperatures = [0, 10, 20, 30, 40]
```



Ejercicio 2

Crear una lista con los números del 1 al 10. Luego crear otra lista modificando la primera aplicándole la siguiente ecuación a cada elemento ' $3n+1$ '. Por último, quedarse solo con los elementos mayores o iguales a 9 y menores que 20.



Ejercicio 3

Modificar el ejercicio del cifrado César usando expresiones lambda y las funciones `ord()` y `chr()`.

Hint: 'Aplicar condicionales en una línea'.

